

Documento Técnico del Sistema Cyclix API

1. Resumen general

Cyclix API es un backend REST en Kotlin + Spring Boot que soporta autenticación JWT, gestión de usuarios, bicicletas, puestos, viajes, soporte, mantenimiento, pricing, suscripciones, wallet, auditoría y analítica administrativa. Usa arquitectura por capas (controller/service/repository/entity-dto), persistencia en MariaDB y migraciones con Flyway.

2. Stack y ejecución

- Kotlin 2.1.20
- Spring Boot 3.4.5
- Spring Data JPA + Flyway
- Spring Security stateless con JWT
- MariaDB
- WebSocket para integración con dispositivos
- OpenAPI/Swagger

Configuración principal:

- ``server.port=6060``
- ``app.jwt.secret``
- ``app.jwt.expiration-seconds``
- ``app.device.api-key``

3. Seguridad y acceso

- Públicos: ``/api/v1/auth/**``, ``/api/v1/device/**``, ``/ws/device``, Swagger.
- GET público en bicicletas (``/api/v1/bicicletas`` y ``/filtrar``).
- Resto requiere JWT.
- Roles: ``USER``, ``ADMIN``, ``MAINTENANCE``.

Flujo JWT

1. Login recibe email/password.
2. Se valida estado ACTIVE y credenciales.
3. Se genera token JWT (HS256).
4. ``JwtAuthenticationFilter`` valida token en requests autenticados.

4. Módulos funcionales

4.1 Auth

Archivos: ``auth/controller``, ``auth/service``, ``auth/dto``, ``auth/security``.

- Registro: crea usuario con rol USER y estado ACTIVE.
- Login: solo usuarios ACTIVE.

4.2 Usuarios

Archivos: ``user/*``.

- Gestión administrativa de usuarios (listar, cambiar estado, asignar rol).
- Perfil autenticado (``/api/v1/profile/me``).

4.3 Puestos

Archivos: ``puesto/*``.

- CRUD de estaciones.
- Listado de puestos activos y con espacio.
- Validación de código único y capacidad.

4.4 Bicicletas

Archivos: `bicycle/\*`.

- CRUD con filtros (estado, tipo, puesto).
- Lectura por QR.
- Baja lógica (`FUERA\_DE\_SERVICIO`).
- Control de capacidad de puesto al mover/crear/retirar bicicleta.
- Coordenadas GPS reales o fallback simulado en Zacapa.

4.5 Viajes

Archivos: `trip/\*`.

- Inicio de viaje: requiere bici DISPONIBLE y sin viaje activo del usuario.
- Finalización: calcula duración, aplica suscripción, tarifa y cobro wallet.
- Admin: listar y cancelar viajes.

4.6 Pricing

Archivos: `pricing/\*`.

- Reglas con prioridad y ventanas por fecha/hora/día/festivo.
- Cálculo de tarifa base + bloques extra.
- Gestión de feriados.

4.7 Suscripciones

Archivos: `subscription/\*`.

- Gestión de planes.
- Asignación de planes a usuarios.
- Consumo de minutos incluidos en viajes.

4.8 Wallet

Archivos: `wallet/\*`.

- Saldo y transacciones del usuario.
- Recarga propia simulada y recarga administrativa.
- Cobro de viaje con validación de saldo (`402` si insuficiente).

4.9 Soporte

Archivos: `support/\*`.

- Tickets generales y reportes de falla.
- Validación de referencias (`bikeId`, `tripId`, `paymentId`).
- Regla de negocio: categoría `EMERGENCY` => prioridad `CRITICAL`.

4.10 Mantenimiento

Archivos: `maintenance/\*`.

- Órdenes manuales o desde ticket BIKE.
- Asignación a técnicos MAINTENANCE.
- Historial de cambios por orden.
- Resolución con tres resultados: AVAILABLE, STAYS_IN_MAINTENANCE, OUT_OF_SERVICE.
- Sincronización de estado de bicicleta y ticket relacionado.

4.11 Dispositivos (ESP32)

Archivos: `device/\*`, `config/DeviceWebSocketConfig.kt`.

- HTTP fallback para unlock.
- Canal WebSocket autenticado por API key y bikeId.
- Publicación de comando `UNLOCK` después del commit de creación de viaje.

4.12 Auditoría y analítica

- Auditoría (`audit/\*`): log de eventos de negocio y endpoint admin de consulta.
- Analítica (`analytics/\*`): métricas de bicicletas, usuarios y estaciones por ventana temporal.

5. Base de datos y migraciones

Migraciones en `src/main/resources/db/migration`:

- V1: usuarios/roles/estados
- V2: soporte
- V3-V4: seed + corrección de contraseñas
- V5: viajes
- V6: puestos/bicicletas
- V7: pricing/suscripción/wallet/audit
- V8: latitud/longitud en bicicletas
- V9: mantenimiento + rol MAINTENANCE
- V10: seed de puestos/bicicletas

6. Reglas críticas del sistema

- No puede existir más de un viaje ACTIVE por usuario.
- Una bicicleta en viaje pasa a EN_USO y al finalizar/cancelar vuelve a DISPONIBLE.
- No se permite cambio de estado en bici FUERA_DE_SERVICIO.
- No más de una orden de mantenimiento activa por bicicleta.
- Cierre de orden con OUT_OF_SERVICE requiere motivo.

7. Pruebas existentes

`src/test/kotlin` incluye pruebas para:

- contexto de aplicación
- bicicleta
- pricing
- suscripción
- wallet
- unlock de dispositivos
- publicación de comandos unlock

8. Hallazgos y mejoras recomendadas

1. Homogeneizar formato de respuesta API (algunos módulos usan `ApiResponse`, otros DTO directo).
2. Agregar paginación en listados grandes.
3. Fortalecer estrategia JWT (refresh/revocación/rotación).
4. Expandir pruebas de integración end-to-end entre viaje+pricing+wallet+suscripción+mantenimiento.
5. Revisar consistencia de documentación y comentarios de migraciones.
6. Alinear Dockerfile/puerto con configuración real de la app.

9. Conclusión

El sistema actual cubre el ciclo operativo completo de micromovilidad

(reserva/uso/cobro/soporte/mantenimiento) con separación clara de responsabilidades por módulo y reglas de negocio bien definidas. Las áreas principales de mejora se concentran en estandarización de API, robustez de seguridad y cobertura de pruebas de integración.